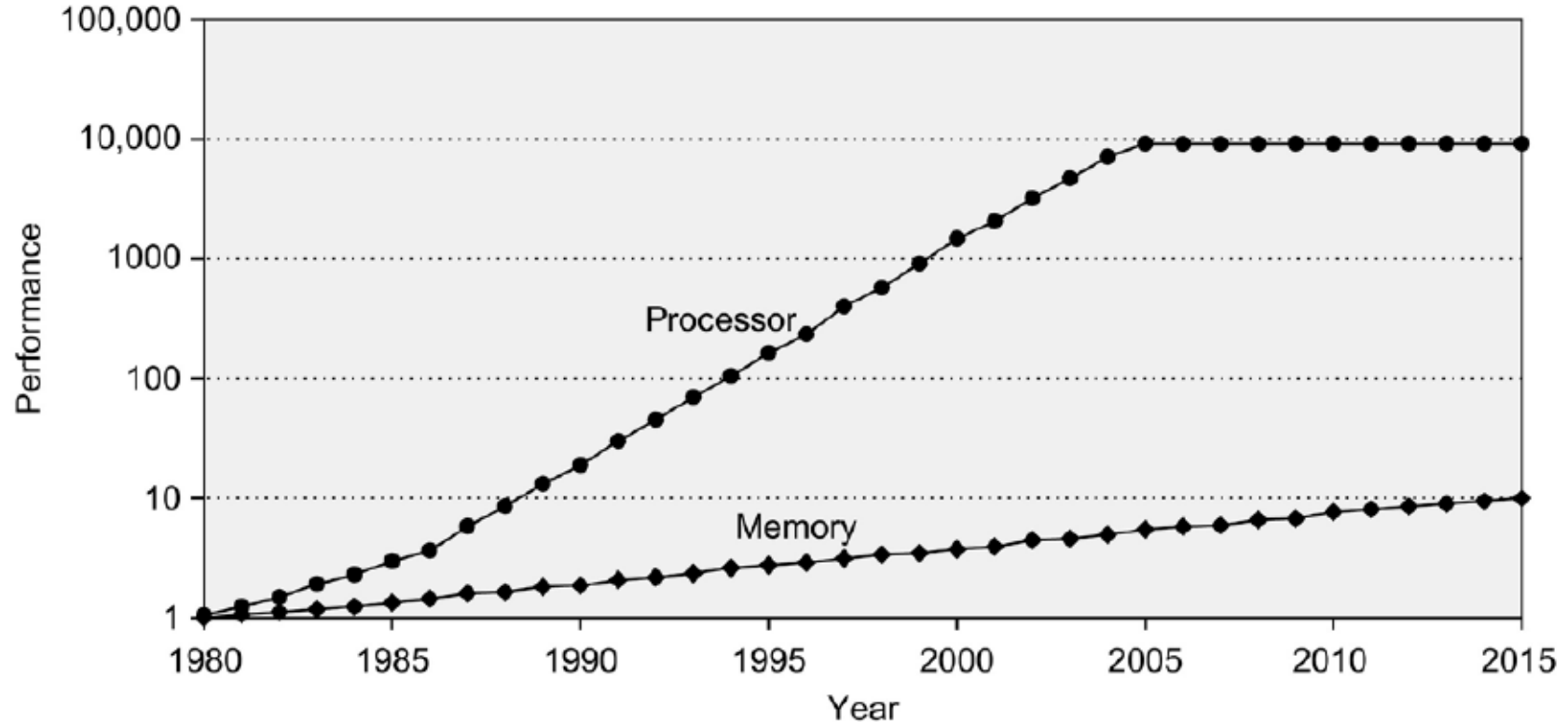


Hierarquia de Memórias

- Conceitos básicos de caches
- Custo de caches
- Desempenho de caches
- Otimizações para caches

Projeto do Sistema de Memórias

- **Desempenho de um programa** depende de:
 - Processador, sistema de memórias, sistema de E/S, ...
- **Arquiteturas multicore**: aumentam n^0 acessos à memória por ciclo
- **Tecnologia**:
 - Memórias rápidas têm maior custo por bit que memórias lentas
 - **Processor-memory performance gap**

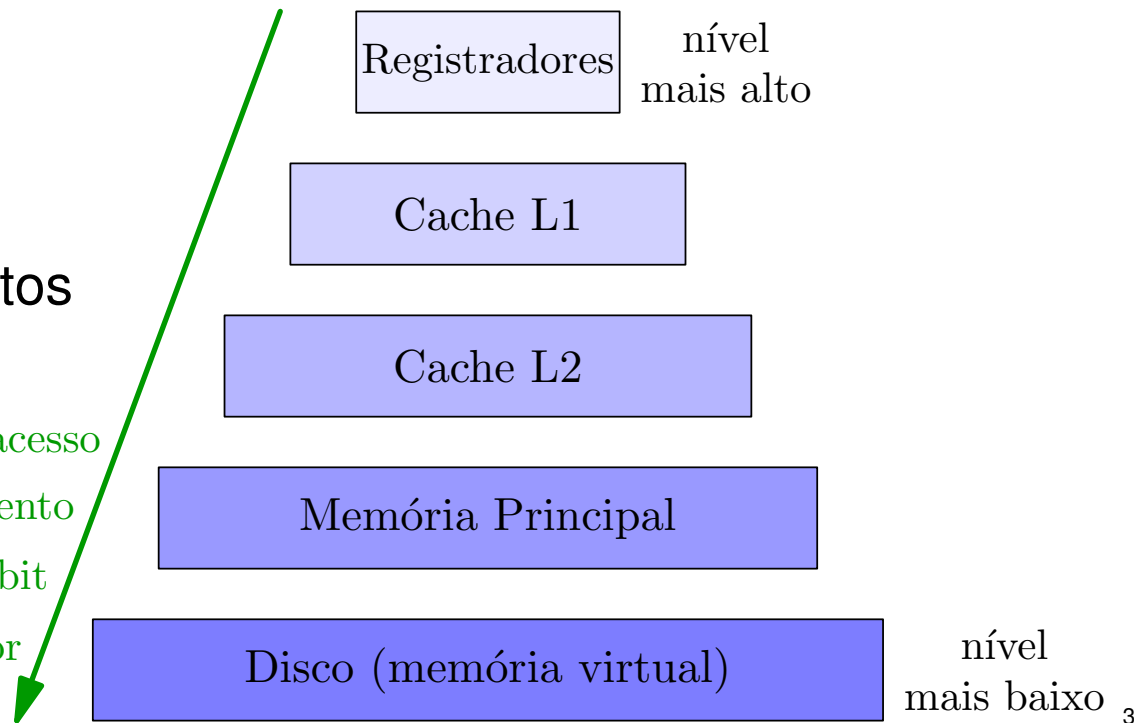


Projeto do Sistema de Memórias

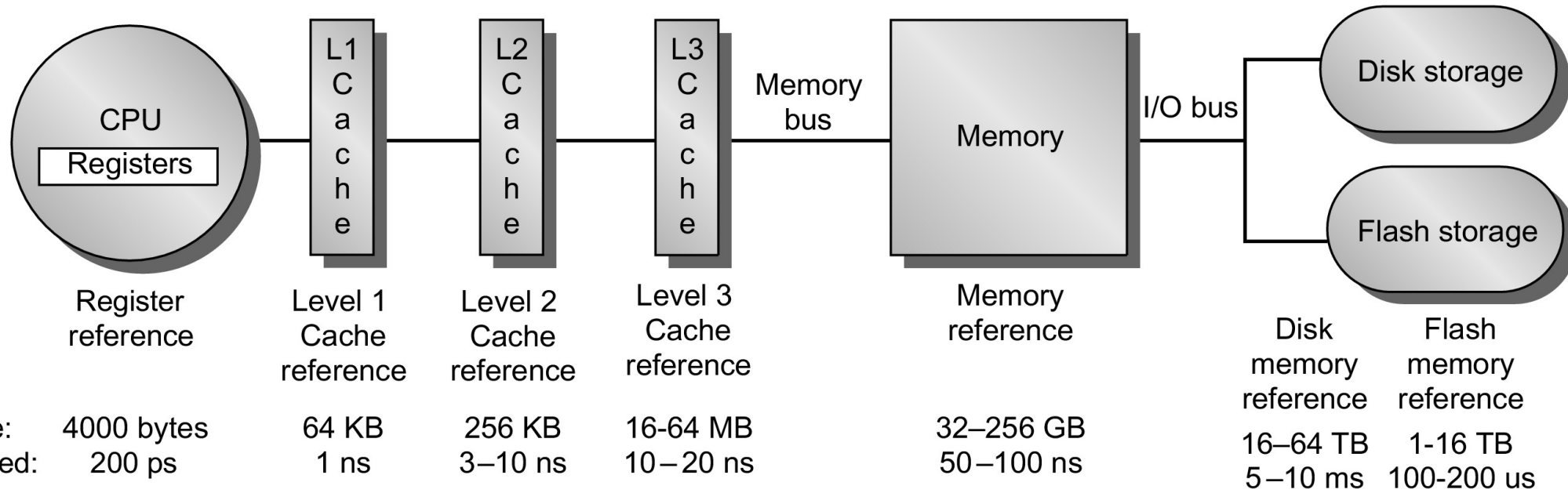
- **Programas:**
 - Precisam de cada vez mais memória, com acesso rápido
 - Em geral, apresentam princípio de localidade
- **Princípio de localidade:**
 - **Localidade temporal**
 - **Localidade espacial**
- **Solução: Organizar sistema de memória como hierarquia**

- Localidade temporal e espacial garantem que quase todas as referências sejam encontradas nas memórias dos níveis mais altos

menor velocidade de acesso
maior capacidade de armazenamento
menor custo por bit
maior distância do processador



Exemplo de Hierarquia de Memórias



Memory hierarchy for server

Exemplo 1: Princípio de Localidade

- **Matriz:** `int a[n][n] ;`

$a_{0,0}$	$a_{0,1}$	$\dots$	$a_{0,n-1}$
$a_{1,0}$	$a_{1,1}$	$\dots$	$a_{1,n-1}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$a_{n-1,0}$	$a_{n-1,1}$	$\dots$	$a_{n-1,n-1}$

- **Armazenamento da matriz na memória principal:**

$a_{0,0}$
$a_{0,1}$
$\vdots$
$a_{0,n-1}$
$a_{1,0}$
$a_{1,1}$
$\vdots$
$a_{1,n-1}$
$\vdots$
$\vdots$
$a_{n-1,0}$
$a_{n-1,1}$
$\vdots$
$a_{n-1,n-1}$

- **Inicialização da matriz em C:**

- **Solução 1:** tempo de execução = 1.3s

```
for (i = 0 ; i < n ; i ++)  
    for (j = 0 ; j < n ; j ++)  
        a[i][j] = i + j ;
```

- **Solução 2:** tempo de execução = 9.5s

```
for (j = 0 ; j < n ; j ++)  
    for (i = 0 ; i < n ; i ++)  
        a[i][j] = i + j ;
```

Memória Cache: Organização e Operação

- **Cache:**

- **Campos:** bit de validade, bit de modificação (dirty bit), tag, bloco
- **Conjuntos**
- **Associatividade**

- **Diferentes organizações:**

- **Cache mapeada diretamente**
- **Cache totalmente associativa**
- **Cache set-associativa**

- **Endereço de memória:**

tag	índice	block offset	byte offset
-----	--------	-----------------	----------------

- **Acerto** (hit) e **falha** (miss) na cache

- **Substituição de blocos:**

- Bloco retirado escolhido dentre todos os blocos **do conjunto**
- **Políticas de substituição:** LRU (Least Recently Used), FIFO, Aleatório

- **Coerência entre cache e MP na escrita:** write-through, write-back

Cache		
V	Tag	Dado

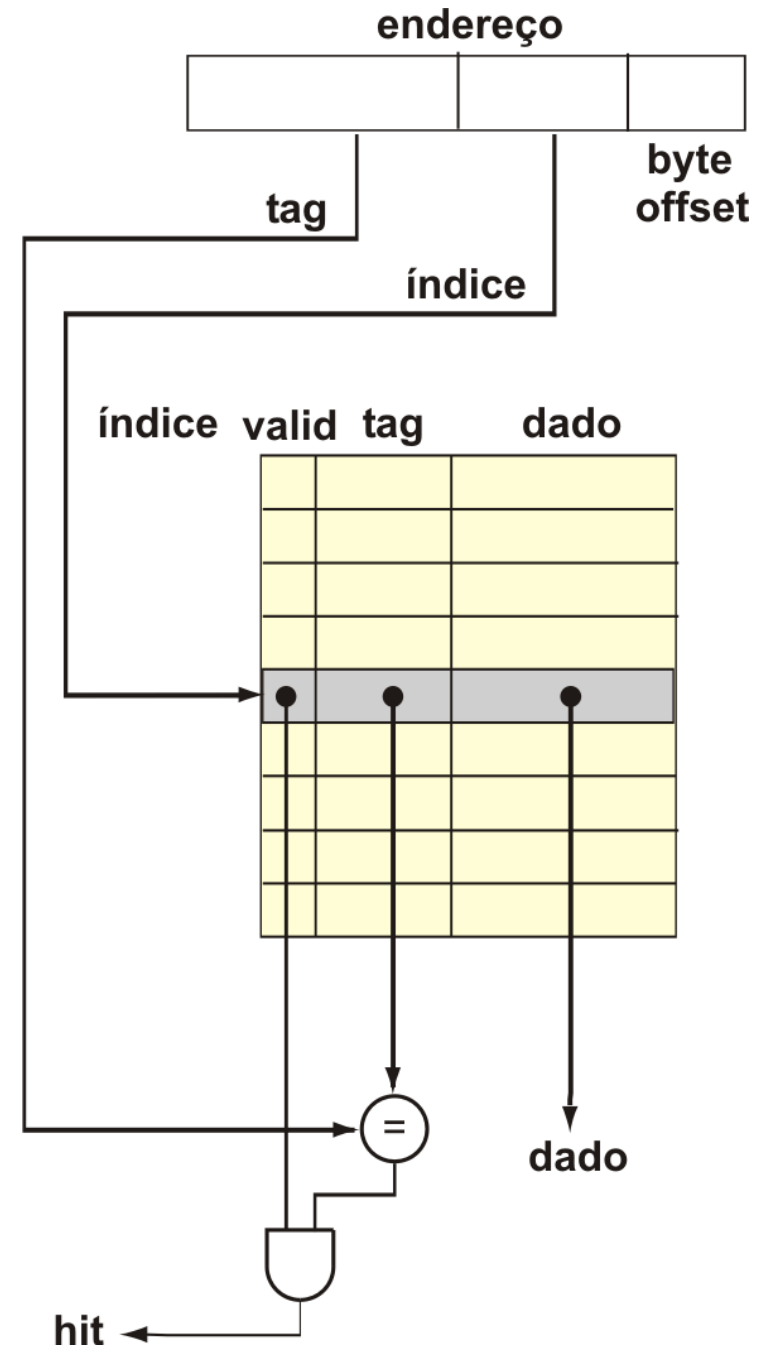
Memória

⋮

Custo do Hardware da Cache

- **Nº total de bits** da cache:
 - Engloba blocos de dados, tags, bits de validade (e outros bits de controle)
 - Depende de:

- **Comparadores** da cache:
 - Nº de comparadores:
 - Depende de:
 - Largura dos comparadores:
 - Depende de:

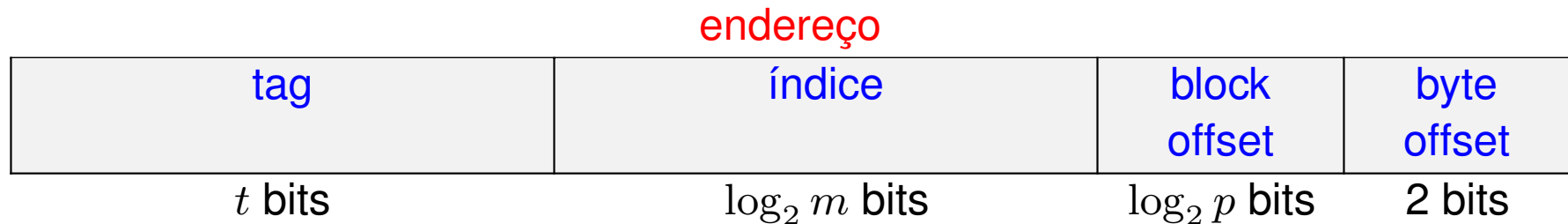


Exemplo 2: Custo do Hardware

- **Cache mapeada diretamente**
- MP endereçada por bytes
- Palavra de 32 bits (4 bytes)
- Tamanho da cache: m blocos
- Blocos de p palavras
- Endereço da MP de 32 bits:

Cache

V	Tag	Dado



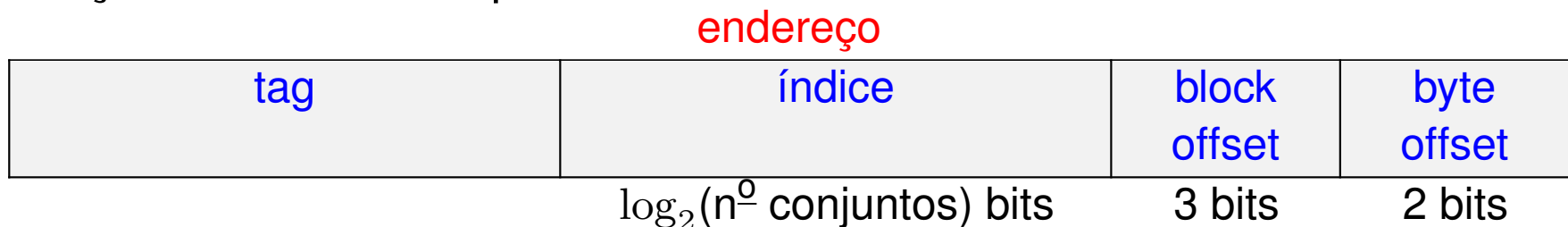
- N^o total de bits da cache = _____
- N^o de comparadores = _____
- Largura dos comparadores = _____

Exemplo 3: Associatividade e Custo do Hardware

- Memória principal endereçada por bytes
 - Cache de 4K blocos = 2^{12} blocos
 - Bloco de 8 palavras de 32 bits
 - N^o total de bits da cache, **excluindo tag**
(apenas V e Dado) =
-

Cache		
V	Tag	Dado

- Endereço de 32 bits: campos



Exemplo 3: Associatividade e Custo do Hardware (continuação)

Cache	<i>mapeada diretamente</i>	<i>2-way set associativa</i>	<i>4-way set associativa</i>	<i>totalmente associativa</i>
Nº de blocos por conjunto (associatividade)				
Nº de conjuntos				
Nº de comparadores				
Nº de bits do índice				
Nº de bits da tag				
Nº total de bits de tag da cache				

Desempenho da Cache

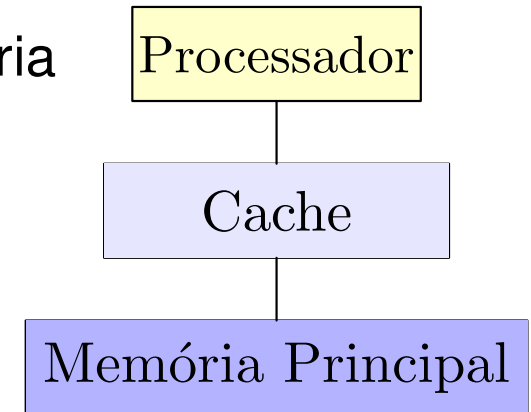
- **Taxa de acertos** (hit ratio): acertos por acesso à memória
 - % dos acessos à memória que causam acertos

$$taxa\ acertos = \frac{n^o\ acertos}{n^o\ acessos_{mem}}$$

- **Taxa de falhas** (miss ratio): falhas por acesso à memória
 - % dos acessos à memória que causam falhas

$$taxa\ falhas = \frac{n^o\ falhas}{n^o\ acessos_{mem}} = 1 - taxa\ acertos$$

- **Tempo de acerto** (hit time):
 - Tempo de acesso ao nível mais alto
(incluindo tempo para determinar se acesso é acerto ou falha)
- **Tempo de tratamento da falha** (miss penalty):
 - Tempo para transferir bloco do nível mais baixo para mais alto e entregar dado ao processador (incluindo tempo de substituição, se necessária)



Desempenho da Cache (continuação)

- **Acessos à memória por instrução**

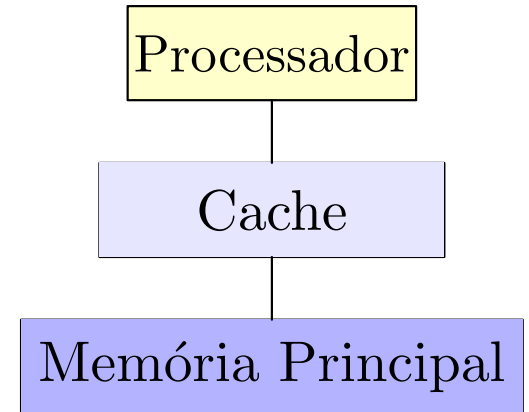
$$acessos_{mem} \text{ por instrução} = \frac{n^{\circ} \text{ acessos}_{mem}}{n^{\circ} \text{ instruções}_{exec}}$$

- **Falhas por instrução:**

$$falhas \text{ por instrução} = \frac{n^{\circ} \text{ falhas}}{n^{\circ} \text{ instruções}_{exec}}$$

- **Tempo efetivo de acesso à memória:**

$$t_{acesso \text{ mem}_{efetivo}} = tempo_{acerto} + taxa \text{ falhas} \times miss \text{ penalty}$$

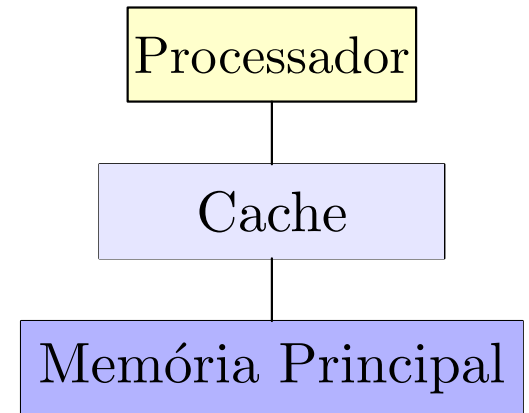


Desempenho da Hierarquia de Memórias

- **Tempo de CPU:**

- **Com cache ideal** (sem falhas):

$$\begin{aligned} t_{cpu} &= n^{\circ} \text{ ciclos}_{cpu} \times t_{ciclo} \\ &= n^{\circ} \text{ instruções}_{exec} \times CPI \times t_{ciclo} \end{aligned}$$



- **Com cache real:**

$$\begin{aligned} t_{cpu} &= (n^{\circ} \text{ ciclos}_{cpu} + n^{\circ} \text{ ciclos}_{stall\ mem}) \times t_{ciclo} \\ &= n^{\circ} \text{ instruções}_{exec} \times (CPI + \text{ciclos}_{stall\ mem\ por\ instrução}) \times t_{ciclo} \end{aligned}$$

- **Número de ciclos em stall pela memória:**

$$n^{\circ} \text{ ciclos}_{stall\ mem} =$$

- **Ciclos em stall pela memória por instrução**

$$\text{ciclos}_{stall\ mem\ por\ instrução} =$$

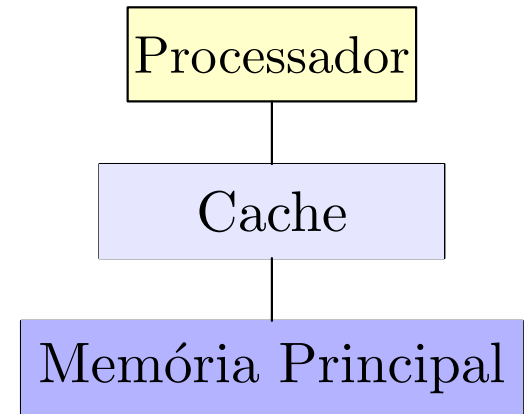
$$=$$

Desempenho da Hierarquia de Memórias

- **Tempo de CPU:**

- **Com cache ideal** (sem falhas):

$$\begin{aligned} t_{cpu} &= n^{\circ} \text{ ciclos}_{cpu} \times t_{ciclo} \\ &= n^{\circ} \text{ instruções}_{exec} \times CPI \times t_{ciclo} \end{aligned}$$



- **Com cache real:**

$$\begin{aligned} t_{cpu} &= (n^{\circ} \text{ ciclos}_{cpu} + n^{\circ} \text{ ciclos}_{stall\ mem}) \times t_{ciclo} \\ &= n^{\circ} \text{ instruções}_{exec} \times (CPI + \text{ciclos}_{stall\ mem\ por\ instrução}) \times t_{ciclo} \end{aligned}$$

- **Número de ciclos em stall pela memória:**

$$n^{\circ} \text{ ciclos}_{stall\ mem} = n^{\circ} \text{ falhas} \times \text{miss penalty em ciclos}$$

- **Ciclos em stall pela memória por instrução**

$$\begin{aligned} \text{ciclos}_{stall\ mem\ por\ instrução} &= \frac{n^{\circ} \text{ ciclos}_{stall\ mem}}{n^{\circ} \text{ instruções}_{exec}} \\ &= \text{falhas por instrução} \times \text{miss penalty em ciclos} \end{aligned}$$

Exemplo 4: Desempenho da Hierarquia de Memórias

- Programa executado em processador:
 - CPI ideal = 2 (sem stall pela memória)
 - Número de instruções executadas = I
 - Porcentagem de `load` e `store` = 30%
- Cache de instruções e dados separadas
- Cache de instruções:
 - Taxa de falhas = 2%
 - Miss penalty = 100 ciclos
- Cache de dados:
 - Taxa de falhas = 4%
 - Miss penalty = 100 ciclos
- **Processador com caches ideais** (sem falhas) (A):
 - $t_{cpu} = \underline{\hspace{15cm}}$

Exemplo 4: Desempenho da Hierarquia de Memórias (cont.)

- **Processador com caches reais (B):**

	Cache de instruções	Cache de dados
Nº de acessos à memória		
Nº de falhas		
Nº de ciclos em stall pela memória		

- Ciclos em stall pela memória por instrução = _____
- CPI real = _____
- t_{cpu} = _____
- **Speedup** de A em relação a B = _____

Classificação de Falhas

- **De acordo com sua causa:**

- **Falha compulsória:**

- Causada pelo 1º acesso a um bloco
- Ocorre mesmo em cache infinita

- **Falha de capacidade:**

- Causada quando cache não pode manter todos os blocos usados por um programa
- Blocos são substituídos e trazidos de volta
- Ocorre mesmo em cache totalmente associativa

- **Falha de conflito ou colisão:**

- Causada quando vários blocos competem pelo mesmo conjunto
- Blocos do conjunto são substituídos e trazidos de volta
- Ocorre mesmo havendo posições vazias em outros conjuntos
- Ocorre apenas em cache mapeada diretamente ou set-associativa
- Eliminada em cache totalmente associativa do mesmo tamanho

Cache		
V	Tag	Dado

Memória

Exemplo 5: Classificação de Falhas

- Cache de 4 blocos, inicialmente vazia
- Acessos aos blocos da MP
- **Cache mapeada diretamente**

Cache			
índice	V	Tag	Dado

Acessos

Endereço	Índice	Falha ?
0	0	
3	3	
4	0	
2	2	
0	0	
1	1	
7	3	
3	3	
7	3	

Exemplo 5: Classificação de Falhas (cont.)

- Cache de 4 blocos, inicialmente vazia
- Acessos aos blocos da MP
- **Cache 2-way set-associativa**
- Política de substituição LRU

Cache			
índice	V	Tag	Dado

Acessos

Endereço	Índice	Falha ?
0	0	
3	1	
4	0	
2	0	
0	0	
1	1	
7	1	
3	1	
7	1	

Exemplo 5: Classificação de Falhas (cont.)

- Cache de 4 blocos, inicialmente vazia
- Acessos aos blocos da MP
- **Cache totalmente associativa**
- Política de substituição LRU

Cache

V	Tag	Dado

Acessos

Endereço	Falha ?
0	
3	
4	
2	
0	
1	
7	
3	
7	